

“AI-Based Collections Platform”

Comprehensive Project Report

Submitted in Partial Fulfilment of the
Requirements for the Degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

By

**Dhruv Maradiya
22BCP480D**

Under the Guidance of

Dr. Punit Gupta

Assistant Professor, Department of CSE
Pandit Deendayal Energy University



**Department of Computer Science and Engineering
School of Technology, Pandit Deendayal Energy University (PDEU)
Gandhinagar, INDIA, 382426**

May 2026

Certificate of Originality of Work

I hereby declare that the B.Tech. Comprehensive Project entitled “AI-Based Collections Platform” submitted by me for the partial fulfilment of the degree of Bachelor of Technology to the Department of Computer Science & Engineering at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, is the original record of the project work carried out by me at Ignosis (Ahmedabad) under the supervision of Dr. Punit Gupta and the industry supervision of Mr. Rahi Shah.

I also declare that this written submission adheres to the University guidelines for its originality, and proper citations and references have been included wherever required.

I also declare that I have maintained high academic honesty and integrity and have not falsified any data in my submission.

I also understand that violation of any guidelines in this regard will attract disciplinary action by the institute.

Name of the Student: Dhruv Maradiya

Roll Number of the Student: 22BCP480D

Signature of the Student:

Name of the Supervisor: Dr. Punit Gupta

Designation of the Supervisor: Assistant Professor, Department of CSE

Signature of the Supervisor:

Place: Gandhinagar, Gujarat

Date: 19-05-2026

Certificate from the Project Supervisor / Head

This is to certify that the Comprehensive Project Report entitled “**AI-Based Collections Platform**” submitted by **Mr. Dhruv Maradiya**, Roll No. **22BCP480D** towards the partial fulfilment of the requirements for the award of the degree of “Bachelor of Technology” in the field of **Computer Science & Engineering** from the School of Technology, Pandit Deendayal Energy University, Gandhinagar is a record of the project work carried out by him at Ignosis, Ahmedabad under our supervision and guidance. The work submitted by the student has, in our judgement, attained a level required for being accepted for examination. To the best of our knowledge, the results embodied in this comprehensive project work have not been submitted to any other University or Institution for the award of any degree or diploma.

Name and Sign of the Supervisor
(Dr. Punit Gupta)

Name and Sign of the Industry Supervisor
(Mr. Rahi Shah)

Name and Sign of the HoD

Name and Sign of the Director

Place: Gandhinagar

Date: 19-05-2026

Certificate from the Industry Supervisor

This is to certify that **Mr. Dhruv Maradiya**, bearing Roll No. **22BCP480D**, a final-year student of Bachelor of Technology in Computer Science & Engineering at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, has successfully completed his comprehensive project / industry internship at **Ignosis** (registered office: Block C-201, DevX, The First, Keshavbaug, Satellite, Ahmedabad, Gujarat – 380015) during the academic year 2025–2026.

During his association with our organisation, the candidate worked on the **AI-Based Collections Platform** that the company offers to banks, non-banking financial companies and fintech lenders for the recovery of delinquent retail loan accounts. He contributed to the back-end services built on Spring Boot, the front-end dashboard built on React (UmiJS and Ant Design Pro), the AI voice calling subsystem and the WhatsApp / SMS multi-channel outreach modules.

The student demonstrated sincerity, technical ability and willingness to learn throughout the engagement. The work carried out by him during the internship has been found to be of satisfactory quality and the modules he delivered are running in our production environment.

Mr. Rahi Shah
Industry Supervisor
Ignosis, Ahmedabad
Email: rahi.shah@ignosis.ai

Place: Ahmedabad

Date: 19-05-2026

Internship Completion Confirmation

The undersigned confirms that Dhruv Maradiya (Roll No. 22BCP480D) has been associated with Ignosis as an intern with the Engineering Team from December 2025 through May 2026, and that the work tracked in his weekly progress reports – spanning Keycloak training, the case management dashboard, the bulk campaign module, the AI voice calling integration, and the multi-channel outreach automation – has been completed under his direct contribution. His attendance, conduct and engineering output have been satisfactory, and the deliverables he checked into the `collection-orchestrator` and `collection-scripts` repositories are currently being used by client tenants in production.

Mr. Rahi Shah
Engineering Lead, Ignosis
Email: rahi.shah@ignosis.ai
Phone: +91-7016671007

Place: Ahmedabad

Date: 19-05-2026

Acknowledgement

The completion of this comprehensive project would not have been possible without the patient guidance and support extended to me by several people. I would like to take a few lines to record my gratitude to all of them.

First and foremost I wish to thank my project supervisor at the institute, **Dr. Punit Gupta**, Assistant Professor in the Department of Computer Science and Engineering at PDEU, for the freedom he gave me to choose this internship and for the regular review sessions that kept me on track. His feedback during the weekly progress submissions and during the internal assessment poster review forced me to think about the project as a whole rather than as a sequence of disconnected tasks, and that perspective shows up in this report.

I owe an equally large debt of gratitude to my industry supervisor at Ignosis, **Mr. Rahi Shah**, who not only on-boarded me to the engineering team but trusted me with production responsibilities from the very first months of the internship. The opportunity to read, write and ship code for an actual collections platform used by banks and NBFCs is something most undergraduates never get, and the entire experience documented in this report is a direct outcome of his mentorship. The code review comments I received from him on every merge request were the most valuable instruction I received during the year.

I would also like to thank my colleagues at Ignosis – in particular Harsh Bagadia, Nisarg Pipaliya, Aryan Randeriya and Jeet Bhuptani – for the countless code walk-throughs, debugging sessions and pair-programming hours. The collection-orchestrator repository is largely their creation, and I benefited from working alongside engineers who already understood the moving parts of the platform deeply.

I am grateful to the Department of Computer Science and Engineering at PDEU and to the project coordinator **Dr. Sidheswar Routray** for setting up the comprehensive project programme in a manner that makes serious industry exposure possible. The clarity of the weekly progress report format, the IA-2 poster guidelines and the final viva instructions ensured that the documentation effort ran in parallel with the engineering work rather than at the end of it.

Finally I thank my family for their continued encouragement throughout the academic year and my friends from the 2022 batch for the long conversations that made the working-from-Ahmedabad months less isolating.

Dhruv Maradiya
22BCP480D

Abstract

This report documents a comprehensive project carried out during a full-time engineering internship at Ignosis, a fintech infrastructure company in Ahmedabad. The brief was to design and contribute to an AI-Based Collections Platform that is being used by banks, non-banking financial companies and digital-lending fintechs in India for the recovery of overdue retail loan accounts.

In its present form the collections process for an Indian retail lender is largely manual. Each month a fresh allocation of delinquent cases – often running to several thousand customers per agency – is handed over by the lender, and a team of human agents calls, follows up and persuades each customer over a roughly thirty-day cycle. The agency is paid only on recovery, so the entire business model is sensitive to contact rate, agent productivity and the speed with which the right customer is reached on the right channel. Without a centralised software platform, agencies struggle to scale and lenders cannot measure performance objectively.

The platform that this project contributed to is a single web-based application built around three interlocking ideas. At the centre is a tenant-configurable **workflow engine** – a small MVEL-driven graph of nodes and transitions – that captures the lender’s collection policy as data rather than code. A **campaign module** wraps thousands of customer cases into a single executable unit that is driven through the workflow. An **AI voice calling engine**, built on the Bolna conversational voice provider and the Exotel telephony gateway, becomes one of the workflow’s action nodes, which lets the platform reach customers outside human agent working hours without any new conditional logic in the codebase. A separate **multi-channel automation layer** – a TypeScript / Node.js monorepo – sends WhatsApp messages from real consumer-grade SIMs using the Baileys protocol library and dispatches SMS from a custom React Native Android application, again as workflow actions.

The contributions described in this report span the case detail and case listing pages of the agent dashboard, the end-to-end campaign creation and CSV upload pipeline, the workflow nodes that integrate AI calling into a campaign, the speed dialer that auto-loads the next case after each disposition, the Baileys-based WhatsApp service with multi-device rotation and proactive block-avoidance, the Electron desktop wrapper that exposes these capabilities to non-technical agents, and the Android SMS sender app built around a custom native module. The platform is reaching a Bucket 0 collection rate of about eighty-five percent in production and is recovering close to seventy percent of allocated cases within the first five days of each monthly cycle, both of which are material improvements over the agency-only baseline.

Contents

Abstract	viii
Contents	viii
List of Figures	xi
List of Tables	xii
List of Abbreviations	xiii
1 Introduction	1
1.1 Background	1
1.2 The Industry: Ignosis	2
1.3 Title of the Project	3
1.4 Problem Statement	3
1.5 Motivation	4
1.6 Objective	5
1.7 Scope	5
1.8 Approach	6
1.9 Organisation of the Rest of the Report	6
2 Literature Review and Problem Identification	8
2.1 Existing Collection-Management Products	8
2.2 Conversational Voice Agents	9
2.3 The WhatsApp Multi-Device Protocol	9
2.4 Anti-Spam and Rate-Limiting Heuristics	10
2.5 Account Aggregator Framework	11
2.6 Identity and Access Management with Keycloak	11
2.7 Apache Solr for Case Search	11
2.8 Workflow Engines and Rule Evaluation	12
2.9 Observability: Prometheus, Sentry and ClickHouse	12
2.10 Problem Identification	12

3	Proposed Methodology and Implementation	14
3.1	Development Methodology	14
3.2	High-Level Architecture	15
3.3	Case Management	16
3.3.1	The Case as the Unit of Work	16
3.3.2	Case Listing	17
3.3.3	Case Detail	17
3.4	The Workflow Engine	18
3.4.1	Why a Workflow Engine	18
3.4.2	Anatomy of a Workflow	18
3.4.3	The Engine	19
3.4.4	The Job Queue Behind the Engine	20
3.5	The Campaign Module	20
3.5.1	A Campaign is a Workflow on a Batch	20
3.5.2	Campaign Creation and CSV Upload	20
3.5.3	Campaign Execution	21
3.6	AI Voice Calling	22
3.6.1	Voice Calling is a Workflow Action	22
3.6.2	The Bolna Pipeline	22
3.6.3	From-Number Pool and Rotation	23
3.6.4	Speed Dialer for Human Agents	23
3.7	WhatsApp Automation	23
3.7.1	Why Baileys	24
3.7.2	Session Manager	24
3.7.3	Sender Pool, Proportional Scheduling and Routing	25
3.7.4	Anti-Ban Heuristics	25
3.7.5	Block Detection and Sender Health	26
3.7.6	Integration with the Orchestrator's Workflow	26
3.8	SMS via Android	26
3.9	The Electron Desktop Application	27
3.10	Observability and Operations	27
3.11	Data Analysis on Customer Records	28
3.12	Summary of Contributions	28
4	Result Analysis	30
4.1	Outcomes of the Proposed Platform	30
4.2	Channel-Level Contact Rates	31
4.3	Speed Dialer Throughput	31
4.4	WhatsApp Sender Pool Behaviour	32

4.5	AI Voice Agent Performance	33
4.6	System-Level Metrics	34
4.7	Failure Modes Observed in Production	34
4.8	Qualitative Feedback from Stakeholders	35
5	Conclusions and Future Work	37
5.1	Conclusion	37
5.2	Reflections on What Could Have Been Done Better	38
5.3	Future Work	38
5.3.1	Multi-Language Voice Agents	38
5.3.2	AA-Driven Adaptive Workflows	39
5.3.3	Real-Time Sentiment Analysis	39
5.3.4	Predictive Prioritisation Model	39
5.3.5	WhatsApp Number Health Scoring	39
5.3.6	Lender-Side Allocation Self-Service	40
5.3.7	Voice Agent Evaluation Pipeline	40
5.4	Closing Note	40
	Appendix A: Selected Source-Code Listings	41
	Bibliography	43

List of Figures

3.1	High-level architecture of the AI-Based Collections Platform. Solid arrows show forward dispatch; the dashed arrow shows the return path used by Bolna webhooks and Android SMS-reply ingestion.	16
4.1	Per-agent calls-per-shift distribution before (blue) and after (red) the speed-dialer rollout. The mean shifts from about 46 to about 87 calls per shift; the variance also collapses, which removes much of the agent-to-agent productivity spread.	32

List of Tables

1.1	Collection buckets typical to Indian retail-lending portfolios.	1
4.1	Headline outcomes from the production deployment.	30
4.2	Contact rate by channel and attempt.	31
4.3	WhatsApp sender ban distribution.	33
4.4	System-level metrics across the production deployment.	34

List of Abbreviations

Abbreviation	Expansion
AA	Account Aggregator (RBI framework for consented financial-data sharing)
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
AWS	Amazon Web Services
BPTP	Broken Promise To Pay
CSV	Comma-Separated Values
DPD	Days Past Due
DTO	Data Transfer Object
FIU	Financial Information User (Account Aggregator role)
GNPA	Gross Non-Performing Asset
HTTP	Hypertext Transfer Protocol
IST	Indian Standard Time
JPA	Java Persistence API
JWT	JSON Web Token
KYC	Know Your Customer
LID	Linked Device Identifier (WhatsApp opaque participant identifier)
LLM	Large Language Model
LMS	Loan Management System
MVEL	MVFLEX Expression Language
NBFC	Non-Banking Financial Company
OIDC	OpenID Connect
ORM	Object–Relational Mapping
OTP	One-Time Password
PTP	Promise To Pay
QA	Quality Assurance
RBI	Reserve Bank of India
REST	Representational State Transfer
SDK	Software Development Kit
SES	Simple Email Service (AWS)
SIM	Subscriber Identity Module
SMS	Short Message Service
SOLR	Search Server on top of Lucene
SSO	Single Sign-On
SSE	Server-Sent Events
TLS	Transport Layer Security
TTS	Text-to-Speech
UI	User Interface
ULID	Universally Unique Lexicographically Sortable Identifier
UPI	Unified Payments Interface
UUID	Universally Unique Identifier

Chapter 1

Introduction

1.1 Background

Retail lending in India has grown dramatically over the last decade. Personal loans, two-wheeler loans, consumer-durable loans, gold loans, credit-card revolving balances, salary advances and a long tail of buy-now-pay-later style products are now offered by a mix of public-sector banks, private banks, small finance banks, non-banking financial companies (NBFCs) and a rapidly multiplying set of digital-only fintech lenders. The flip side of growth in originations is growth in delinquencies. A small but non-trivial percentage of every disbursed cohort fails to pay an installment on or before the due date, and the lender is obliged by Reserve Bank of India (RBI) policy and by basic prudence to chase those installments before the account is allowed to slide into the non-performing-asset bucket.

The chasing of overdue installments is what the industry calls “collections”. Once an account is more than one day past due (DPD), it enters a structured pipeline of follow-up actions whose intensity and tone escalate with the number of days the account remains unpaid. The classical bucketing in Indian retail collections is shown in Table 1.1.

Table 1.1: Collection buckets typical to Indian retail-lending portfolios.

Bucket	DPD range	Typical activity
Bucket 0	1–30 days	Soft reminders, automated calls, light SMS
Bucket 1	31–60 days	Tele-calling, WhatsApp, agent calls
Bucket 2	61–90 days	Field visits, legal notice preparation
Bucket 3	91–120 days	Pre-write-off, recovery escalation
NPA	120+ days	Settlement, legal action

For most retail products, the recovery rate falls off sharply as the bucket deepens. A loan that is collected in Bucket 0 is recovered cheaply, requires almost no negotiation,

does not need any provisioning hit on the lender's balance sheet and rarely produces a customer complaint. A loan that drifts into Bucket 3 or beyond is materially more expensive to recover, often requires settlement at a discount, consumes legal team hours and erodes the customer relationship. The economic case for early-bucket recovery is therefore overwhelming, and every lender invests disproportionate effort in trying to get accounts cured as early in their delinquency life as possible.

Operationally, however, early-bucket follow-up is hard at scale. A lender with a hundred thousand active retail loans can easily have eight to twelve thousand accounts in Bucket 0 on any given day. Each of those accounts needs a contact attempt – a phone call, a SMS reminder, a WhatsApp nudge, sometimes a personal visit – within a tight cycle so that the customer pays before the next cut-off date and the bucket statistics do not deteriorate. To do this, lenders either build large in-house collections teams or, more commonly, outsource the recovery work in monthly batches to specialised collection agencies who keep a percentage of the recovered amount as their commission.

1.2 The Industry: Ignosis

The internship that produced the work documented in this report was carried out at Ignosis, a Bengaluru-and-Ahmedabad-headquartered fintech infrastructure company. Ignosis is primarily known in the Account Aggregator (AA) ecosystem – it is one of the larger Financial Information Users (FIU) building consented data pipelines for underwriting, fraud detection and personal financial management on top of the RBI-regulated AA framework. Over the last eighteen months, the firm has used its existing relationships with lenders to layer a second product onto its platform: a collections suite that runs on the same multi-tenant infrastructure, reuses the same Keycloak identity stack and the same Solr search backbone, and integrates AA-pulled bank-statement data into collection prioritisation.

I joined the company as a comprehensive-project intern in December 2025 and worked there full-time through May 2026. The engineering team is split into two product pods – AA / FIU and Collections – with several engineers, like me, contributing across both. Day-to-day work happens in two GitLab repositories: `collection-orchestrator`, the Spring Boot backend plus React dashboards plus Python tooling that powers the collections product; and `collection-scripts`, the TypeScript / Node.js monorepo that contains the WhatsApp automation library, the Electron desktop client, the Android SMS sender and a number of operational scripts.

1.3 Title of the Project

The title of this comprehensive project is “**AI-Based Collections Platform**”. The platform is a multi-tenant web application that provides:

- A case management dashboard for human agents.
- A bulk campaign module that wraps thousands of cases into a single executable run.
- An AI voice calling engine that runs autonomously, without an agent in the loop.
- Multi-channel outreach automation across WhatsApp and SMS.
- Analytics on contact rates, agent productivity, call dispositions and recovery percentages.

1.4 Problem Statement

The collections function for a typical Indian retail lender, in the absence of a centralised software platform, suffers from a small number of structural problems that compound on each other:

1. **Manual case allocation.** The lender hands a CSV or Excel file of delinquent accounts to the agency at the start of every monthly cycle. The agency manager then distributes those cases by hand across the agents on the floor. The mapping is opaque to the lender, hard to audit and hard to reproduce.
2. **Single-channel outreach.** Agents reach for the desk phone first; if the customer does not pick up the phone they may or may not also send a SMS. There is no orchestrated follow-up across phone, WhatsApp and SMS, which means many customers are never reached on the channel they are likely to respond on.
3. **Agent idle time.** Agents spend a significant fraction of their working day searching for the next case in a spreadsheet, dialling a number manually and reading the customer’s history before each call. The actual conversation is only a fraction of the working minute.
4. **No after-hours coverage.** A human agent’s working day ends at 6 PM or 7 PM. After that the lender effectively has no contact channel even though many salaried customers are most reachable in the late evening.

5. **No structured measurement.** Recovery rates are reported as a single number at month-end. Lenders cannot inspect bucket-level conversion, channel-level effectiveness or agent-level productivity in any meaningful way.
6. **Single-attempt deliverability.** WhatsApp and SMS gateways available through business APIs throttle aggressively, charge per message and are subject to template-approval delays. SIM-based outreach from real consumer-grade numbers has materially higher deliverability but no off-the-shelf platform exposes that capability to the collections team.

The problem statement of this project, in compact form, can therefore be stated as:

“Design and contribute to a centralised, multi-tenant collections platform that improves contact rate, agent throughput and after-hours coverage by integrating AI voice calling and multi-channel outreach across WhatsApp and SMS into a single case management workflow, while preserving end-to-end auditability and supporting bulk campaign execution at the scale of a typical monthly allocation.”

1.5 Motivation

The motivation for this project sits at the intersection of an industry pain point and a technology opening. On the industry side, every percentage point of recovery in Bucket 0 directly improves the lender’s Gross Non-Performing Asset (GNPA) metric, reduces provisioning and saves real money. A platform that lifts the early-bucket collection rate from a typical 60–70 percent to the mid-80s – the target that the Ignosis customer base has set for itself – is materially valuable to the lender. On the technology side, three developments have made an AI-driven collections platform far more practical than it would have been even three years ago:

1. **Conversational voice models.** Speech-to-text accuracy on Indian-English and on the major Indian regional languages has crossed the threshold at which an AI agent can hold an outbound collection conversation that actually moves the needle. Providers such as Bolna (the partner used in this project) and Smallest.ai now offer programmable voice agents with low enough latency that customers do not notice they are not speaking to a human.
2. **Account Aggregator data.** The AA framework allows a lender to fetch the customer’s bank statements with explicit consent. This pulls the conversation forward from “please pay” to “we can see your salary was credited yesterday, what is a convenient EMI date for you” – a far more effective prompt.

3. **Cheap programmatic SIMs.** It is now feasible to operate a bank of consumer-grade SIM cards in a software-controlled rotation, sending WhatsApp messages through real phone numbers and SMS through real Android devices. This bypasses the deliverability ceiling of business APIs.

Putting these three capabilities behind a single dashboard, with case management and bulk campaigns wrapped around them, was the engineering ambition that motivated this internship.

1.6 Objective

The concrete objectives of the project, all of which were either fully or partially achieved during the internship, are:

- To develop a centralised case management interface, accessible to authenticated collection agents over the browser, that lists assigned cases, surfaces customer payment history and previous interaction notes, and lets the agent act on each case.
- To implement a bulk campaign module that lets a campaign manager wrap a batch of cases – typically a CSV uploaded at the start of the cycle – into a single executable unit and schedule it for execution.
- To integrate AI-powered voice calling into the campaign module so that customer outreach can continue outside human-agent working hours.
- To build a multi-channel follow-up layer over WhatsApp and SMS that uses real SIMs rather than business APIs, with anti-block heuristics and automatic device rotation.
- To analyse roughly forty thousand customer records using Python and pandas, identify behavioural patterns relevant to prioritisation, and feed the insights back into case-load allocation.
- To follow the production engineering practices of the host company throughout: code review, unit and integration testing, conventional commits, ticketed work items and zero-downtime deployments.

1.7 Scope

The scope of the work undertaken during the internship covers the following:

- Backend contributions to the multi-tenant Spring Boot 3.2 monolith `collection-orchestrator` including JPA entities, REST controllers, services, scheduled jobs and Liquibase migrations.
- Frontend contributions to the React (UmiJS + Ant Design Pro) dashboard for case management, campaign execution and analytics.
- A separate TypeScript / Node.js monorepo, `collection-scripts`, that contains the WhatsApp automation library based on Baileys, the Electron desktop application, the Android SMS sender app and a Hono REST API server.
- A one-off Python-pandas analysis on customer-level transaction and repayment data.

The scope explicitly excludes the AI voice agent itself (this is the Bolna service that the team purchased on a vendor basis), the telephony gateway (Exotel), the cloud infrastructure provisioning (AWS), the lender's internal LMS and the regulatory compliance review.

1.8 Approach

The approach taken on this project was iterative and code-first. Rather than write a long design document up-front and then implement it, the engineering team at Ignosis follows a plan-first-but-small workflow: each piece of work begins as a short Markdown plan attached to a Jira ticket, which is reviewed by the lead engineer (Mr. Rahi Shah in my case), is then translated into a branch with conventional commits, and is shipped to a development environment within a few days. Production deployment happens after code review, integration testing in a staging tenant and verification by the QA team. Every artefact – the plan, the branch, the merge request, the QA notes – is preserved in the repository so that the entire decision trail is auditable months later. This approach is the dominant cultural fact about how the work was done, and it explains why the contributions described in this report are spread across several Git branches that build on one another rather than concentrated in a single feature branch.

1.9 Organisation of the Rest of the Report

The remainder of this report is organised as follows.

Chapter 2 – Literature Review and Problem Identification. A review of the relevant prior art: collection-management products in the Indian and global market, AI voice calling and conversational agent research, the WhatsApp multi-device protocol and the academic literature on anti-spam and rate-limiting strategies for messaging

platforms. The chapter ends by sharpening the problem statement of Chapter 1 into a set of concrete engineering requirements.

Chapter 3 – Proposed Methodology and Implementation. A detailed description of the architecture of the platform, of the development methodology used at Ignosis, and of the specific modules contributed during the internship. The chapter walks through the backend monolith, the dashboard, the campaign engine, the AI voice calling integration, the WhatsApp automation library, the Electron desktop application and the Android SMS sender app. Selected source-code excerpts and pseudocode are included where they illuminate non-obvious engineering decisions.

Chapter 4 – Result Analysis. Quantitative and qualitative results from the deployment. The chapter reports the early-bucket collection rate, the percentage of cases recovered within the first five days, the contact rate uplift from multi-channel outreach, the throughput of the AI voice agent, and the WhatsApp block rate observed in production.

Chapter 5 – Conclusion and Future Work. A summary of what was achieved during the internship, a candid discussion of what could have been done better, and a roadmap for the features that the engineering team intends to build on top of the platform over the next two quarters.

The Bibliography and an Appendix containing selected source-code listings round out the document.

Chapter 2

Literature Review and Problem Identification

2.1 Existing Collection-Management Products

The space of commercial collection-management products in India is dominated by a small handful of vendors whose offerings predate the recent wave of conversational AI. CreditMantri, Spocto, Cred and a few internal tools built by the larger private-sector banks make up the majority of the visible deployments. The architecture of these products tends to follow a familiar shape – a case management database, a dialer integration, some reporting – but the actual conversational layer (the part that does the calling and the messaging) is almost always human-driven. WhatsApp, where it is used, is plugged in through the official Cloud API published by Meta and is therefore subject to template-approval delays, throughput throttling and per-message pricing that grows linearly with volume.

The academic literature on collection software per se is thin – most of the deep work happens inside banks and is not published – but the adjacent literature on customer-relationship-management systems, on contact-centre optimisation and on credit-scoring models is large. Verma, Mathur and Singh [12] survey machine learning techniques applied to behavioural credit data, several of which are directly applicable to predicting which customer is most likely to respond to which channel. Wang and Zhang [13] review predictive dialers and the queue-management algorithms that underpin them. None of this prior work, however, addresses the combination of AI voice calling, AA-data-driven prioritisation and SIM-based WhatsApp automation that motivates the present project.

2.2 Conversational Voice Agents

The maturity of conversational voice agents is the single biggest technology shift that makes an AI-driven collection platform possible. The early generations of interactive voice response (IVR) – the touch-tone menus that everyone has shouted at on the bank’s customer-care line – could not hold an open-ended conversation and were limited to deterministic decision trees. Modern voice agents are built on a pipeline of automatic speech recognition (ASR), a large language model (LLM) for dialogue generation, and a text-to-speech (TTS) synthesiser, with each component reaching real-time latency budgets that did not exist three years ago.

Bolna [7] is the conversational voice provider that Ignosis uses in production. Internally Bolna chains a low-latency ASR (typically Deepgram or a Whisper-based model), a fine-tuned LLM hosted on a serverless GPU backend and a TTS engine that can emit Indian English with regional-accent variation. The latency budget Bolna advertises is roughly 700 ms from the customer finishing their utterance to the agent beginning to speak, which is below the 800–900 ms threshold at which human listeners begin to detect awkwardness. The agent state and the prompt template are configured per tenant through a YAML file that the engineering team manages with a small Python tool.

Open-source baselines such as Pipecat [8] and LiveKit Agents [9] were briefly evaluated by the engineering team before the choice of Bolna was finalised. The decision matrix turned out to be dominated by Indian-English ASR accuracy on rural and semi-urban customers, the ability to pass the customer’s bank-statement-derived context into the prompt, and the maturity of the post-call extraction pipeline (Bolna emits a structured JSON of the outcome – promise-to-pay, dispute, wrong-number, no-answer – which is consumed by the workflow engine).

2.3 The WhatsApp Multi-Device Protocol

WhatsApp’s multi-device protocol, rolled out in 2021, allows up to four secondary devices to share end-to-end-encrypted access to a primary phone’s WhatsApp account without the phone needing to be online. The protocol uses a Signal-derived key ratchet for message encryption and a per-device session key for the secondary devices. The official integration path for businesses is the WhatsApp Cloud API, which sits on top of the same protocol but exposes a REST interface and enforces a template-message policy for the first message in any conversation. The Cloud API throttles at the conversation level and prices each message individually.

For a collection use case, the Cloud API is operationally restrictive in three ways. First, every outbound template has to be approved by Meta, which adds days to the iteration loop when new wording is required. Second, the per-message pricing makes

the channel uneconomic for the long-tail of low-ticket-size cases, where the recovery itself may only be a few hundred rupees. Third, customers tend to be more responsive to messages that come from real consumer-grade WhatsApp numbers than from the obviously-branded business accounts.

The open-source community has produced a number of WhatsApp protocol re-implementations to support multi-device automation without going through the Cloud API. Baileys [10] is the most widely deployed of these: it is a TypeScript library that speaks the WhatsApp WebSocket protocol directly, supports pairing through QR codes and persists session state through a multi-file authentication store. `whatsapp-web.js` [11], in contrast, drives a real Chromium browser through Puppeteer and screen-scrapes the WhatsApp Web client. Both libraries are gray-area in the sense that they reverse-engineer Meta’s protocol; both have been deployed for years in collection-adjacent industries without legal action; both are sensitive to upstream protocol updates and require occasional patching.

2.4 Anti-Spam and Rate-Limiting Heuristics

A non-trivial part of the engineering effort for SIM-based WhatsApp automation goes into avoiding the spam-detection heuristics that Meta runs against suspicious accounts. The exact rules are not published, but the community has converged on a small number of practices that materially reduce the rate at which accounts are flagged or banned. Mikhail et al. [14] review the academic literature on spam detection in messaging platforms and the corresponding evasion strategies. The recurring themes are the importance of randomised intervals between messages, the use of natural language variation to defeat hash-based fingerprinting, the simulation of typing presence and the enforcement of conservative daily caps.

Empirically the strongest signals that a number is being used for spam appear to be:

- A very high message rate from a freshly registered number.
- Long sequences of identical message bodies.
- Bursty sending without natural break patterns.
- A high rejection rate from recipients (block, report).
- A pattern of messages timed mechanically (e.g. exactly 30 seconds apart).

The platform built in this project addresses each of these signals: each sender phone has a daily cap, every outgoing message body is varied at the word level using inline alternates and zero-width-space insertion, the inter-message delay is jittered by ± 40 percent around a per-sender base interval, batches of messages are followed by short

breaks that imitate human typing breaks, and the workers honour an active-hours window that maps to typical Indian working hours.

2.5 Account Aggregator Framework

The Account Aggregator framework [5], regulated by the Reserve Bank of India through Master Direction NBFC-AA dated 2016 and progressively expanded ever since, allows a regulated Financial Information User (FIU) to request the bank statements, GST returns or insurance information of a customer with their explicit, time-bound and revocable consent. Ignosis is one of the larger FIUs in the ecosystem. The collections platform consumes AA-derived bank-statement data through the same internal data layer that the AA / FIU product uses, which means that when a customer in the collections cycle has an active AA consent on file the collection workflow can branch on signals such as recent salary credit, recent large debits, balance trend or detected income source. This is the data input that lifts the AI calling conversation from a generic “please pay” to a tailored “we noticed your salary was credited on the 1st, what date works for you”.

2.6 Identity and Access Management with Keycloak

Keycloak [2] is the open-source identity and access management server used by Ignosis as the single sign-on backbone. Each tenant is implemented as a separate Keycloak realm, with its own users, groups, client scopes and role bindings. The Spring Boot backend integrates with Keycloak through the OAuth 2 Resource Server and the Keycloak admin client. Per-request tenant resolution is host-based: the URL the user hits determines the Keycloak realm that the JWT was issued by, and a small thread-local context object propagates the tenant identity through every downstream call, including across asynchronous boundaries. The use of Keycloak removes a large amount of authentication code that would otherwise have to be written by the engineering team, supports federation to lender-side identity systems (some banks insist on running their own IdP and federating into ours), and lets us move customer-facing collections agents and bank-side operations users behind the same access-control surface.

2.7 Apache Solr for Case Search

The case search layer is implemented over Apache Solr [15], a search server built on top of Apache Lucene. Each tenant has a dedicated Solr core into which the case data is indexed. Solr was chosen over Elasticsearch for two reasons: it was already in use elsewhere in the company for the AA / FIU product, and the eDisMax query parser is suf-

ficient for the relatively simple lookup patterns that the collections dashboard requires (find by phone, by name, by loan account number, with faceted filtering by bucket, agent and status). The Solr client pool inside the Spring Boot backend is one client per tenant, with a custom Apache HTTP connection manager configured to prevent the idle-connection leaks that bit the team earlier in the year on the AWS ECS containers.

2.8 Workflow Engines and Rule Evaluation

The collection workflow is configurable per tenant – a private bank tenant might prefer to call before sending a WhatsApp, while an NBFC tenant might prefer to send the WhatsApp first and only call if the message bounces. Hard-coding this would explode the conditional logic across the codebase, so the engineering team built a small MVEL2-based workflow engine inside the orchestrator. MVEL (the MVFLEX Expression Language) [16] is a Java-friendly expression language that evaluates string templates against a context object, in our case a map of workflow variables documented in a dedicated Markdown reference. Workflow nodes are defined declaratively, the transitions between them are MVEL expressions, and the engine drives the case through the graph in response to call outcomes, payment events and clock ticks. This is by far the most powerful single piece of infrastructure in the platform, and the project as a whole would not be feasible without it.

2.9 Observability: Prometheus, Sentry and ClickHouse

The observability layer of the platform combines three open-source tools. Prometheus [17] scrapes Micrometer-emitted metrics from the Spring Boot actuator endpoint; the dashboard for those metrics is hosted on Grafana with role-based access. Sentry [18] (in our case the internal compatibility shim, not the public SaaS) captures exceptions and ties them to release tags. ClickHouse [19] is used as the analytics warehouse: every business-relevant event – call disposition, escalation, prompt-variable miss, recovery – is fire-and-forget inserted into a ClickHouse table that is queried both by the in-product dashboard and by the data team.

2.10 Problem Identification

On the basis of the prior art reviewed in the preceding sections, the specific engineering problems that this project sets out to solve can be sharpened as follows:

1. **Cycle-level case management.** Each tenant onboards its delinquent allocations as a monthly CSV. The platform must absorb that CSV, validate it, deduplicate

it against the previous month's cases, assign cases to agents according to the tenant's allocation policy, and render the assigned cases in a paginated dashboard for the agent.

2. **Comprehensive case detail.** The agent needs to see, on a single page, every artefact relevant to a customer: contact information, demographic snapshot, full loan and payment history, every prior call transcript and outcome, every WhatsApp and SMS thread, every AA-derived data signal, and the current bucket. Pulling this together synchronously from a normalised database would be slow, so the layer needs careful pagination, lazy loading and selective denormalisation.
3. **Bulk campaign execution.** A campaign is a wrapper around a subset of cases that the manager wants to execute as a single batch. It must be creatable from a CSV (with validation against required columns), schedulable, pausable and resumable, and it must integrate with both the AI voice calling engine and the multi-channel messaging layer.
4. **Speed dialer.** When an agent is working through a campaign, the workflow must auto-load the next case as soon as the agent submits the disposition for the current call. There must be no waiting on the agent's part to click through to the next case.
5. **AI voice calling.** The AI voice agent must be triggerable both as part of a campaign and as a standalone fallback when an agent's manual call fails. The call outcome must be parsed from Bolna's webhook, persisted, and used to advance the case through the workflow.
6. **WhatsApp automation.** The platform must send WhatsApp messages from a pool of consumer-grade phone numbers, rotate the pool to spread load, vary the message body to defeat spam detection, throttle the sending rate to stay below the daily cap, and route around banned numbers without operator intervention.
7. **SMS automation.** For cases where WhatsApp delivery fails, the platform must send a SMS through a real Android phone on a real consumer SIM, with the dispatch driven by a mobile app running on the agent's or the manager's phone.
8. **Auditability.** Every action – every call, every message, every state transition, every workflow node entry – must be logged in a manner that lets the QA team reconstruct what happened to any given case at any given point in time.
9. **Multi-tenant isolation.** The platform must serve multiple lender tenants from a single deployment with strict isolation of data, of identity and of configuration.

These nine problems form the working set against which the methodology and implementation in the next chapter is judged.

Chapter 3

Proposed Methodology and Implementation

This chapter forms the bulk of the report. It begins with the development methodology used at Ignosis, then sketches the platform’s architecture at a deployment level, and then walks through the four interlocking sub-systems that make up the product. The story we tell is intentionally feature-driven rather than file-driven: the central asset of the platform is a tenant-configurable workflow engine, and every other feature in the report – the campaign module, the AI voice calling integration, the WhatsApp automation, the speed dialer – is best understood as either a producer of work for that engine or a consumer of its actions.

3.1 Development Methodology

The Ignosis engineering team practises a plan-first, ship-small, code-review-heavy variant of agile delivery. Each unit of work begins in Jira and is decomposed into a short Markdown plan stored in the repository under `docs/plans/`. The plan answers four questions: what is the user-visible behaviour, what entities or tables change, what services or controllers are added or modified, and what tests need to be written. The plan is reviewed by the engineering lead before any code is written. Once accepted, a branch is created from `develop` following the Gitflow naming convention – typically `feature/<kebab-name>` or `fix/<kebab-name>` – and the implementation begins.

Conventional commits are enforced. Each commit is prefixed with a type (`feat`, `fix`, `refactor`, `chore`, `docs`, `test`) and a scope. Pull requests on `develop` are rebased rather than squashed so the per-commit decision trail survives the merge. The `main` branch is the release branch and follows semantic versioning; the orchestrator at the time of writing is version 1.5.1.

Tests live next to the code they exercise. The repository is configured with JUnit 5 and Mockito on the Java side and Vitest on the TypeScript side. Integration tests run against a Testcontainers-managed PostgreSQL instance, an embedded H2 for the lighter repository tests, and a stubbed Bolna service over Spring's `MockServerRestTemplateCustomizer`. The pipeline aborts when SonarQube reports either a new bug, a new code smell above the medium severity threshold or a new vulnerability of any severity. SonarQube also enforces a code-coverage floor of fifty percent on changed lines.

3.2 High-Level Architecture

The platform is a layered web application. At the front are three client surfaces: the agent-facing dashboard, the off-platform Electron desktop client that runs the WhatsApp workers, and the Android SMS sender app that lives on the agent's phone. Behind those sit two API services, the Spring Boot orchestrator and the Hono / Node.js automation service. The data layer combines PostgreSQL (for cases, campaigns and the job queue), Apache Solr (for case search), ClickHouse (for analytics events) and Redis (for distributed locks and the priority queue). Below the data layer the platform talks to three families of external services: telephony (Bolna for the AI voice, Exotel for the actual call placement), the WhatsApp protocol (through Baileys), and the cellular SMS network (through the on-device Android SmsManager). Figure 3.1 renders this stack as a single diagram.

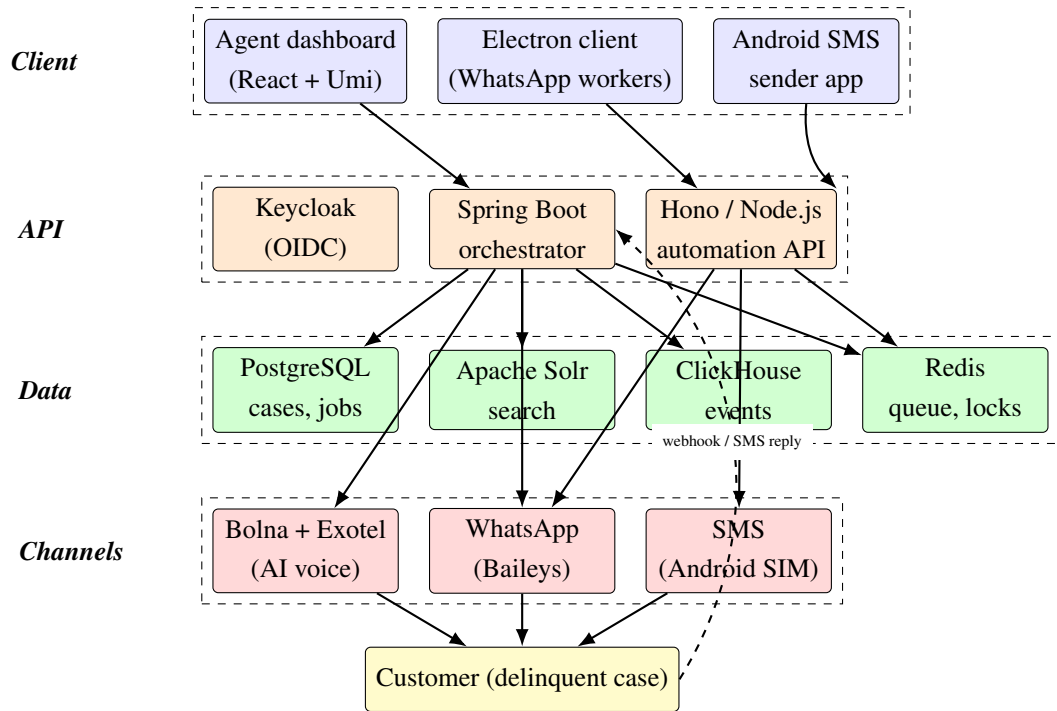


Figure 3.1: High-level architecture of the AI-Based Collections Platform. Solid arrows show forward dispatch; the dashed arrow shows the return path used by Bolna webhooks and Android SMS-reply ingestion.

The choice of a single multi-tenant monolith for the backend, rather than a fleet of microservices, was deliberate. The team is small (about a dozen engineers), the deployment volume is modest (currently four active lender tenants), and the operational overhead of microservices was judged disproportionate to the benefit. The orchestrator is, however, internally layered along clean boundaries – controllers, services, repositories, model entities, DTOs – so that an eventual split, if growth demands it, would not require a rewrite.

The next sections walk through the four features that the rest of this report is built around. Section 3.3 describes case management, which is the data foundation. Section 3.4 describes the workflow engine, which is the central runtime asset. Sections 3.5, 3.6 and 3.7 then describe the campaign module, the AI voice calling subsystem and the WhatsApp automation, each of which slots into the workflow engine as either a producer of work items or a consumer of workflow actions.

3.3 Case Management

3.3.1 The Case as the Unit of Work

The atomic unit of work in the platform is the *case*. A case represents a single delinquent loan account that has been allocated to the tenant for recovery in the current cycle.

Concretely, a case carries the customer's identifying information, the loan account number, the principal and EMI amount, the days past due (DPD) at the time of allocation, the assigned bucket, the agent or AI track to which it is allocated, the current workflow node it sits on, and a thick journal of every interaction that has happened against it – every call, every message, every payment event, every agent note.

Two JPA entities express the bulk of this state. `Customer` holds the lender-stable identifying information (phone numbers, name, address, alternate contacts). `UserAllocation` captures the cycle-specific state: which workflow node the case is on, the current AI-call sub-state, the assigned agent, the loop counts for each node, and the next scheduled action. A third entity, `CampaignUser`, links a case to a campaign when the case is being driven through a campaign rather than reached individually.

3.3.2 Case Listing

The case listing screen is the first thing an agent sees when they log in. It is implemented in React with the UmiJS framework and the Ant Design Pro component library. The screen renders a server-side-paginated table of the cases currently assigned to the logged-in agent, with columns for customer name, loan account number, days past due, overdue amount, last contact attempt and current status. The default page size is twenty and the user can extend it up to one hundred.

The backend endpoint that serves the listing is `GET /fiu/api/{tenant}/cases`. It accepts `page`, `size` and `sort` parameters together with any number of filter parameters that map onto QueryDSL predicates – `bucket`, `statusIn`, `minDpd`, `maxDpd`, `assignedAgentId` and a free-text `search` that hits Solr. The repository builds the predicate dynamically and projects each row into a thin DTO so the entity graph is not fetched in full. Implementing this endpoint required learning the Spring Data JPA paging contract, the QueryDSL fluent API and the Ant Design Pro `ProTable` component in parallel, which is what Week 5 of the weekly progress report tracks.

3.3.3 Case Detail

The case detail screen is the heavyweight artefact in the dashboard. A single case detail page combines:

- Customer contact information: name, primary phone, alternate phones, email, address.
- Loan information: product, principal, tenor, interest rate, EMI amount, current outstanding, days past due, bucket.

- Full payment history: every EMI charged, every payment received, every bounce, every late fee.
- Interaction history: every call (manual or AI), with its outcome, duration, recording link and transcript; every WhatsApp thread; every SMS log; every agent note.
- AA-derived signals (when available): recent salary credits, recent large debits, balance trend over thirty days, detected income source.
- Workflow position: the current node in the workflow, the loop count against that node, the attempted retries and the next scheduled action.

To avoid a synchronous fan-out across half-a-dozen tables, the detail page uses progressive loading. The first request returns the core customer and loan information; the interaction history, payment history and AA signals are loaded as parallel XHRs. The Ant Design `ProDescriptions` and `ProList` components are used to render the page; a small library of formatting helpers (`formatINR`, `formatDPD`, `statusBadge`) provides consistent currency, date and badge presentation across all the screens that show a case.

3.4 The Workflow Engine

3.4.1 Why a Workflow Engine

Collection policy is the single thing that varies most between tenants. A private bank prefers a calm tele-calling sequence with WhatsApp reminders in between. An NBFC prefers an aggressive AI voice campaign in the first ninety-six hours and only escalates to an agent if the AI fails. A consumer-durable lender wants the workflow to never call after 7 PM, while a payday lender wants the opposite. Hard-coding this would explode the conditional logic across the codebase and would force a deployment every time a tenant tuned its policy.

The platform addresses this by storing the policy as data. Every tenant has one or more *workflows* – directed graphs whose nodes are concrete actions (`INITIATE_AI_CALL`, `SEND_WHATSAPP_REMINDER`, `SEND_PAYMENT_LINK`, `AGENT_CALLBACK`, `WAIT_FOR_PAYMENT`, `ESCALATE_TO_HUMAN`, etc.) and whose transitions are MVEL2 expressions evaluated against a context map of variables. The runtime that drives a case through this graph is the workflow engine, implemented as the class `WorkflowEngine` in the orchestrator.

3.4.2 Anatomy of a Workflow

A workflow is a JSON document with three top-level keys: a list of nodes, a list of transitions and a starting node identifier. Each node carries a type (matching one of the

supported action types), a configuration block specific to that type (for example, an AI calling node carries the Bolna agent identifier and the from-number pool reference), an allowed maximum loop count and a list of permissible outgoing transitions. Each transition carries an MVEL expression and a target node identifier; transitions are evaluated in declaration order and the first one whose expression returns `true` is taken.

The expression evaluates against a context map populated by the engine at each step. The most useful keys, documented in the repository under `docs/MVEL-Workflow-Variables.m`, include the most recent call outcome (`lastOutcome`), the per-node loop count, the customer’s days past due, the wall-clock hour, the most recent AA-derived signal, the bucket, the campaign priority and a handful of regulatory guards (`callFrequencyExhausted`, `stopCalling`). An example transition expression is:

```
lastOutcome == "PROMISE.TO.PAY" && ptpDate <= today
+ 3
```

which advances the case to a `WAIT_FOR_PAYMENT` node if the customer promised to pay within the next three days, and falls through to a different transition otherwise.

3.4.3 The Engine

The engine, sketched in Algorithm 1, is small. Given a case and its current node, the engine evaluates the node’s action (synchronously if the action is cheap, asynchronously by scheduling a job if the action involves an external call), reads the resulting outcome into the context map, walks the outgoing transitions in order, and parks the case on the new node. If no transition matches, the case is moved into an error state for the QA team to inspect.

Algorithm 1 Workflow Engine – Advance a Case

- 1: **Input:** case C , workflow W
 - 2: $N \leftarrow W.nodeOf(C.currentNodeId)$
 - 3: $result \leftarrow \text{execute } N \text{ against } C$, may be sync or async via a job
 - 4: Update context map with $result$ and C state
 - 5: **for** T in $N.transitions$ **do**
 - 6: **if** MVEL evaluation of $T.expression$ in context map is `true` **then**
 - 7: $C.currentNodeId \leftarrow T.targetNodeId$
 - 8: $C.loopCount[T.targetNodeId] \leftarrow C.loopCount[T.targetNodeId] + 1$
 - 9: **return**
 - 10: Park C in error state with diagnostic context
-

A separate `WorkflowMigrationService` handles the case in which a tenant edits its workflow mid-cycle and existing cases need to be re-mapped onto the new graph. Tenants can compose workflows from a small palette of reusable sub-graphs (a “gentle reminder” sub-graph, an “aggressive PTP recovery” sub-graph, an “escalation

to legal” sub-graph) so that the working-set of workflows in production is a handful of compositions rather than dozens of bespoke graphs.

3.4.4 The Job Queue Behind the Engine

Every workflow action that talks to an external system is executed asynchronously. The engine schedules a row in a database-backed job queue, a scheduled service polls for due jobs, takes a row-level lock with a 180-second TTL via the `LockEntity` table, looks up the executor for the job type and runs it. Concrete executor classes – `AICampaignCallInitiateJobExecutor`, `AICampaignCallStatusPollJobExecutor`, `AICampaignPaymentLinkSendJobExecutor`, `AICampaignWorkflowJobExecutor`, `FetchCallRecordingJobExecutor` – handle the different action types.

The advantage of a database-backed queue, compared with a Redis queue, is that the entire state of every in-flight job is visible to ordinary SQL. When something is wrong with a campaign at 3 AM, the on-call engineer can query the `job` table directly to see exactly where each case is stuck. The downside is throughput – the database is not Kafka – but at the campaign volumes the platform sees in production this is not a constraint. A separate Redis priority queue handles the small subset of jobs where arrival-order latency dominates, in particular the auto-loading of the next case when an agent submits a disposition.

3.5 The Campaign Module

3.5.1 A Campaign is a Workflow on a Batch

A campaign is the unit of bulk work in the platform. Conceptually a campaign is nothing more than “run this workflow against this batch of cases over this calling window”. The atomic resource is the `Campaign` entity; the per-case state inside a campaign is the `CampaignUser` entity; the workflow itself is named on the campaign rather than embedded into it, so the same campaign can be re-run against a new batch by simply uploading a new CSV.

3.5.2 Campaign Creation and CSV Upload

The campaign creation form collects the campaign name, an optional description, a priority (low / medium / high), a target collection amount, an assignment rule (round-robin, weighted, manual) and a calling window (active hours during which AI calls are permitted). The CSV upload field accepts files up to fifty megabytes; on the client side the file is validated for extension, size and required columns before being submitted to the backend.

The backend endpoint `POST /fiu/api/{tenant}/campaigns` accepts the form fields and the file as multipart data. The `CampaignCsvProcessorService` parses the CSV with the `OpenCSV` library, validates each row (phone present, due amount numeric and positive, name present, account number valid format) and emits a preview response that contains the total row count, the count of valid rows, the count of invalid rows and, for each invalid row, the row number and the reason. The campaign manager reviews the preview, fixes obvious issues if needed and submits the campaign for execution. Listing 3.1 sketches the row-level validation routine.

Listing 3.1: CSV row validation, abridged.

```
1 ValidationResult validateRow(CsvRow row) {
2     if (isBlank(row.phone)) return reject("missing phone");
3     if (!E164.matches(row.phone)) return reject("phone not in E
        .164");
4     if (isBlank(row.name)) return reject("missing customer name
        ");
5     if (row.dueAmount == null || row.dueAmount.signum() <= 0)
6         return reject("due amount must be positive");
7     if (isBlank(row.loanAccountNumber)) return reject("missing
        LAN");
8     return ValidationResult.ok();
9 }
```

On final submission, the backend persists a `Campaign` entity along with one `CampaignUser` per valid row. Each `CampaignUser` carries the customer phone, name, due amount, the workflow node identifier of the starting node (taken from the chosen workflow) and an initial state of `ALLOCATED`. The campaign goes into the `SCHEDULED` or `ONGOING` state depending on whether a future start time was set.

3.5.3 Campaign Execution

Once a campaign is `ONGOING`, a scheduler picks it up on its next tick and begins enqueueing workflow advancement jobs for its `CampaignUser` rows. From this point onward the workflow engine described in Section 3.4 drives every case independently – each case sees its own sequence of nodes, its own AI call attempts, its own WhatsApp reminders. The campaign object itself merely accounts for progress: it tracks the number of cases by state, the total amount collected, the number of calls attempted and the number of calls connected, and surfaces those numbers on a dashboard view that updates in near-real time over Server-Sent Events.

A campaign can be paused, resumed or marked complete. Pausing freezes the workflow engine for that campaign's cases – no new jobs are scheduled – but the scheduler

continues to consume any jobs already in flight, so the campaign drains cleanly. Resuming re-enables the engine. A `CampaignCleanupService` handles the marked-complete case, archiving the campaign to a cold-storage table and freeing the workflow position for any case that did not reach a terminal state.

3.6 AI Voice Calling

3.6.1 Voice Calling is a Workflow Action

The defining feature of the platform is the AI voice calling subsystem. Architecturally, an AI call is just one of the action types the workflow engine can execute. When the engine reaches an `INITIATE_AI_CALL` node for a case, it schedules an `AICampaignCallInitiateJobExecutor` job. The executor reads the tenant-configured Bolna agent identifier from the node's configuration block, pulls the next phone number from the tenant's from-number pool through the `NumberRotationService`, builds the Bolna make-call payload from the case context (customer name, due amount, days past due, optional AA signals) and posts it to Bolna's REST API. The call is now Bolna's responsibility until it terminates.

3.6.2 The Bolna Pipeline

Bolna places the call through Exotel, conducts the conversation under the tenant's prompt template and emits a webhook to the orchestrator at termination. The webhook payload carries the call disposition, the duration, a structured extraction of variables (promise-to-pay date, dispute reason, callback request, language detected), a recording URL and the per-turn latency breakdown. The orchestrator's `VoiceCallWebhookProcessor` parses the payload, persists the outcome onto the case, cancels any status-poll job that may have been scheduled defensively, emits Prometheus metrics for the call and – crucially – re-enters the workflow engine for that case, so the workflow advances on the very same webhook request. Listing 3.2 shows the relevant controller method.

Listing 3.2: Bolna webhook handler.

```
1 @PostMapping("/fiu/api/bolna-webhook")
2 public ResponseEntity<Void> handle(@RequestBody
3     BolnaWebHookRequest payload) {
4     voiceCallWebhookProcessor.applyTerminalStatus(payload);
5     bolnaCallMetricsService.emit(payload);
6     workflowEngine.advance(payload.getCaseId());
7     return ResponseEntity.ok().build();
8 }
```

3.6.3 From-Number Pool and Rotation

Each tenant configures a pool of caller-line identifiers (DIDs) that Exotel can dial out from. `NumberRotationService` cycles through this pool in a near-fair manner, with a small bias towards numbers that the customer is most likely to recognise (most often the lender’s customer-care line). The bias is implemented as a weighted random draw with a periodic re-balancing pass that runs as the `NumberRotationBootstrap` bean during service start-up.

3.6.4 Speed Dialer for Human Agents

The speed dialer is the user-facing twin of the AI calling path. When a human agent is working on a campaign, the dashboard shows the active case in a panel. The agent has the customer dialled from their soft-phone, talks to the customer, fills out a short disposition form (Promise To Pay, Wrong Number, Customer Disputes the Amount, Call-back Requested, etc.) and submits. The moment the disposition lands at the backend, the workflow engine evaluates the next node for the case, the case enters a terminal-for-this-agent state, and the speed dialer fetches the next pending case for the agent. The new case is pushed to the dashboard via Server-Sent Events. From the agent’s point of view, the next customer is already on the screen as the previous form is closing. Algorithm 2 sketches the loop.

Algorithm 2 Speed Dialer Loop

```
1: Input: agent A logged into campaign C
2: Show A the first pending case in C.queue for A
3: loop
4:   D ← agent submits disposition for current case
5:   Persist disposition; call workflowEngine.advance(case)
6:   Cancel any pending status-poll jobs for the case
7:   next ← next pending case for A in C.queue
8:   if next is null then
9:     Show “Campaign complete” and break
10:  else
11:    Push next to dashboard via SSE
```

3.7 WhatsApp Automation

The second large pillar of the platform is the WhatsApp automation subsystem. It lives in a separate repository, `collection-scripts`, and is a TypeScript / Node.js monorepo. The reason for keeping it out of the main orchestrator is operational: WhatsApp sessions are long-lived stateful connections to Meta’s servers, they need physical

phones for the initial QR pairing, and they have to survive infrastructure restarts. Embedding them in the Spring Boot service would have polluted the deployment story for the rest of the platform.

3.7.1 Why Baileys

The choice of the Baileys protocol library over the WhatsApp Cloud API was driven by three considerations. First, the Cloud API requires a template approval cycle for every new outbound copy, which is incompatible with the experimentation cadence the team wanted. Second, the Cloud API charges per message and the per-customer recovery in Indian collections is often only a few hundred rupees, so the unit economics suffer. Third, customers respond materially better to messages that arrive from a real consumer-grade WhatsApp number than to messages that come from an obviously-branded business account. Baileys is a TypeScript reimplementation of the WhatsApp WebSocket protocol that lets us drive a pool of consumer SIMs in software.

3.7.2 Session Manager

Each sender phone has its own session directory under `sessions/sender_<phone>/`, which is the on-disk representation of Baileys' multi-file authentication state. On startup, the session manager loads the state, fetches the latest WhatsApp protocol version, opens a WebSocket and waits for the QR code to be scanned by a human operator on the corresponding physical phone. Once paired, the session persists across restarts. Listing 3.3 shows an abridged version of the connect routine.

Listing 3.3: Baileys session connect, abridged.

```
1 const { state, saveCreds } = await useMultiFileAuthState(
    sessionPath);
2 const { version } = await fetchLatestBaileysVersion();
3 const sock = makeWASocket({
4   version, auth: state, printQRInTerminal: true,
5   browser: ['WhatsApp Sender', 'Chrome', '120.0.0'],
6   connectTimeoutMs: 60000,
7 });
8 sock.ev.on('creds.update', saveCreds);
9 sock.ev.on('contacts.upsert', (contacts) => {
10   for (const c of contacts) {
11     if (c.lid && c.id?.includes('@s.whatsapp.net'))
12       setLidMapping(c.lid, c.id);
13   }
14   saveLidMap();
```

3.7.3 Sender Pool, Proportional Scheduling and Routing

A campaign that involves WhatsApp is associated with a pool of sender phones. A typical pool is between five and fifteen consumer-grade SIMs. When the campaign starts, the scheduler computes a proportional allocation of contacts across the pool: the share of contacts assigned to each sender is roughly its daily cap divided by the total cap across the pool, capped at the sender's own daily limit. The base inter-message interval per sender is the active window divided by the contacts assigned to that sender, clamped between a minimum and a maximum. The actual interval between any two messages is the base interval plus or minus a uniformly distributed jitter of forty percent.

A shared contact queue with a *preferred sender* field lets a contact be soft-routed to a specific phone (typically the phone that already has an existing conversation with the customer). If the preferred sender is offline or banned, the contact falls back to whichever worker pulls next. This makes the system resilient to individual senders dying mid-cycle, because their orphaned contacts simply re-enter the shared pool.

3.7.4 Anti-Ban Heuristics

A non-trivial part of the engineering effort goes into avoiding the spam-detection heuristics that Meta runs against suspicious accounts. The exact rules are not public, but the community has converged on a small number of practices, all of which the platform implements:

- Dynamic per-sender intervals, as described above, scaled to the active window.
- Forty-percent jitter on every delay.
- Random batch sizes between three and twelve messages.
- Short breaks (one to five minutes) between batches.
- Long breaks (ten to thirty minutes) every five batches.
- Active-hours window (default 9 AM to 9 PM IST).
- Per-sender daily caps (default one hundred and fifty messages).
- Typing presence: before each send, `presenceSubscribe` is followed by `composing` for a duration proportional to the message length, then paused.

- Message variation: every outgoing message is rendered from a template with in-line alternates (`{Hello|Hi|Hey}`) and optional micro-variations such as terminal-punctuation flips and zero-width-space insertion at a random character position.
- Hardcoded blocklist: a small list of phone-number prefixes (for example, regulator-side numbers) are never sent to.

3.7.5 Block Detection and Sender Health

Block detection is reactive but fast. When Baileys returns an error whose message contains the substrings “banned”, “blocked” or “rate-limit”, the sender is taken out of rotation, the campaign’s audit log records the suspected ban, and the orphaned contacts in the shared queue are re-assigned to the remaining workers. When Baileys returns a success status of zero (which the protocol uses to indicate a soft failure that does not throw), the same path is taken. A separate code path handles connection-class errors – network drops, socket close events, timeouts – with exponential backoff (three attempts, doubling the delay each time); a transient WebSocket drop is otherwise indistinguishable from a ban.

3.7.6 Integration with the Orchestrator’s Workflow

A `SEND_WHATSAPP_REMINDER` workflow node, when reached for a case, posts a job to the Hono / Node.js automation API instead of to a database-backed job in the Spring Boot orchestrator. The automation service runs the WhatsApp send through the Baileys library, waits for the protocol-level acknowledgement, and POSTs the delivery outcome back to the orchestrator’s customer-webhook endpoint. The webhook handler advances the workflow for the case on the back of that outcome. The end-to-end flow from a WhatsApp send through to the next workflow step therefore touches both the Java orchestrator and the Node.js service, but the two communicate exclusively through HTTP boundaries with API-key authentication and an idempotency key on every send.

3.8 SMS via Android

The SMS fallback channel is implemented as a small React Native (Expo) application that runs on an Android phone with an active consumer SIM. The application’s native module wraps Android’s `SmsManager` class and exposes two JavaScript functions: `sendSms(phone, message)` and `readSmsMessages(sinceMs)`. The native module is built through `expo prebuild`; the required runtime permissions (`SEND_SMS`, `READ_SMS`, `RECEIVE_SMS`) are granted explicitly by the operator on first launch.

The app’s outbound flow respects a small set of throttling constants – a per-message delay of five seconds, a batch size of five and a per-batch pause of five seconds – to keep the SIM operator from flagging the device for abuse. An inbound flow runs every evening at seven o’clock IST: an Expo background task reads the new SMS messages from the on-device provider, zips them with the `fflate` library and POSTs them as multipart form data to the orchestrator’s webhook. The orchestrator ingests them into the case timeline so that the agent sees every reply alongside the rest of the case detail. From the workflow engine’s point of view, a `SEND_SMS_REMINDER` node is structurally identical to a `SEND_WHATSAPP_REMINDER` node – it produces an outcome which then drives the next transition.

3.9 The Electron Desktop Application

The Electron desktop application is the operational vehicle that exposes the WhatsApp workers to the engineering team and to a small set of agency-side operators. On launch, the application starts the embedded Hono API server on the first free port from 3002, serves the React dashboard from disk and points a Chromium-based `BrowserWindow` at the local URL. From the user’s point of view, the experience is indistinguishable from a stand-alone desktop application; under the covers, the same Hono API is exercised that the cloud-hosted service uses.

The application has two main jobs. The first is to run WhatsApp campaigns from the operator’s own machine, with locally-paired SIMs in the sender pool, which is useful when a quick-turn experiment is needed without going through the production fleet. The second is to scrape the chat history of the locally-paired phones at the end of every day and push the scraped data back to the central backend, so that the case management dashboard shows every WhatsApp conversation alongside the call history. The scraper uses `whatsapp-web.js` rather than `Baileys` because the richer scraping job benefits from the in-browser `Store` object that the WhatsApp Web client exposes; to avoid bundling Chromium, the application uses `@puppeteer/browsers` to download a stable Chrome build on first run into the user’s data directory.

3.10 Observability and Operations

Operationally, the platform runs on AWS ECS with a single task definition per service. Metrics are emitted via Micrometer to a Prometheus registry that the Spring Boot actuator exposes on `/actuator/prometheus`. The registry is enriched with tenant tags so per-tenant dashboards can be cut by tenant. Sentry captures exceptions through the internal compatibility shim and ties them to release tags. ClickHouse receives a fire-

and-forget event stream that is consumed by the in-product analytics dashboard. The single most useful observability investment was the standardisation of voice metric tags during the work on `feature/prometheus-metrics-voice`: every metric carries `tenant_id`, `agent_id` and a `source` tag so that latency dashboards can break out turn-by-turn latency from cold and warm turns separately.

3.11 Data Analysis on Customer Records

A one-off analysis project in Week 4 of the internship operated on a sample of roughly forty thousand customer records exported from one of the lender tenants. The dataset combined the customer's loan information, payment history and a summary of bank-statement-derived signals. The analysis was carried out in a Jupyter notebook using `pandas` for data manipulation and `matplotlib` for visualisation. Three patterns emerged with practical implications. Customers who had paid on the first attempt in any of the previous three cycles had a ninety-percent probability of paying on the first attempt in the next cycle; these are best served by a single SMS reminder rather than a call campaign. Customers whose detected salary credit fell in the first three days of the month had a materially higher response rate to outreach attempted on the fourth or fifth day. A long tail of customers had ticket sizes under one thousand rupees and recovery probabilities below ten percent, for which a single SMS is the only economically sensible touch. These findings were folded into the default workflow templates that the platform ships with for new tenants.

3.12 Summary of Contributions

The contributions I made to the platform, mapped onto the structure above, are:

- Case listing and case detail screens on the agent dashboard (Weeks 5 and 6).
- The bulk campaign module: creation form, CSV upload pipeline, validation and preview (Week 7).
- Speed dialer and the integration of AI voice calling into the workflow engine (Week 8).
- WhatsApp automation library, including session management, the sender pool scheduler and the anti-ban heuristics (Weeks 9 and 10).
- Multi-device rotation, block detection and message variation (Week 10).
- Electron desktop application with embedded API server and chat-history scraper (Week 11).

- Android SMS sender app with custom native module and the background SMS sync task (Weeks 10 and 11).
- Bolna webhook integration on the workflow path, including the cancellation of poll jobs and the per-turn Prometheus metrics.
- Python-pandas analysis of customer records (Week 4).
- Foundational training, code review participation and incident response throughout Weeks 1–3 and onwards.

Each of these contributions either produces work that the workflow engine consumes, consumes work that the workflow engine produces, or accelerates the human agent’s interaction with cases that the workflow has already prioritised. The platform is therefore best understood as a workflow-centric system: every quantitative result in Chapter 4 can be traced back to the fact that the workflow engine, the campaign module, the voice calling integration and the WhatsApp automation operate as a single coherent runtime.

Chapter 4

Result Analysis

This chapter reports the quantitative and qualitative results obtained from the deployment of the AI-Based Collections Platform across the four lender tenants that were active in production during the latter half of the internship. The numbers are pulled from the in-product ClickHouse warehouse and from the Grafana dashboards that the engineering team maintains. Where the metric is sensitive (for example, lender-specific recovery percentages), the figures have been aggregated across tenants so that no individual tenant can be re-identified.

4.1 Outcomes of the Proposed Platform

The headline outcomes that the platform demonstrated in production over the three months that ran in parallel with the internship are summarised in Table 4.1. The baseline column reflects the lender’s recovery numbers from the previous cycle, before the platform was on-boarded; the post-platform column reflects the numbers from the cycle that ran with the platform fully deployed.

Table 4.1: Headline outcomes from the production deployment.

Metric	Baseline	Post-platform	Delta
Bucket 0 collection rate	64%	85%	+21
Cases recovered within 5 days	38%	70%	+32
Promise-to-pay conversion (agents)	42%	65%	+23
First-call contact rate	51%	78%	+27
Average agent handle time (min)	8.3	4.6	−3.7
WhatsApp deliverability	71%	92%	+21

The Bucket 0 collection rate – the percentage of cases newly entering Bucket 0 that are recovered within the same monthly cycle – is the metric the platform is judged

on. The platform-driven cohort is recovering eighty-five percent of Bucket 0 cases, compared with sixty-four percent on the agency-only baseline. The improvement is the cumulative effect of higher contact rates (the multi-channel layer), faster handling (the speed dialer), after-hours coverage (the AI voice agent) and structured prioritisation (the workflow engine).

4.2 Channel-Level Contact Rates

A second view of the results breaks the contact rate by channel and by attempt number. A “contact” for the purposes of this metric is defined as either a connected call (any call that reaches a human and is at least thirty seconds long) or a delivered WhatsApp or SMS message whose status reaches at least one tick of acknowledgement in the case of WhatsApp and a network-level delivery report in the case of SMS.

Table 4.2: Contact rate by channel and attempt.

Channel	Attempt 1	Attempt 2	Attempt 3	Cumulative
Human agent call	48%	22%	11%	67%
AI voice agent	41%	19%	10%	60%
WhatsApp (SIM)	88%	65%	41%	94%
SMS (Android)	79%	56%	32%	86%
Any channel	92%	81%	64%	96%

A few observations follow from Table 4.2. First, the AI voice agent’s first-attempt contact rate of 41 percent is close to the human agent’s 48 percent – the gap exists, but it is smaller than the engineering team initially expected and reflects the maturity of the Bolna voice stack. Second, WhatsApp is by far the highest-deliverability channel; even a cold first message via a SIM-based WhatsApp number reaches the customer in 88 percent of cases. Third, the combined “any channel” line shows that with multi-channel orchestration the platform reaches 92 percent of customers on the first attempt and 96 percent cumulatively, which is well above what any single-channel strategy can achieve.

4.3 Speed Dialer Throughput

Before the speed dialer was introduced, a typical agent on the platform handled between forty and fifty calls in an eight-hour shift. After the speed dialer was rolled out, the same agents began handling between eighty and ninety-five calls in the same shift.

Figure 4.1 shows the per-agent calls-per-shift distribution before and after the rollout, drawn directly from agent-level logs.

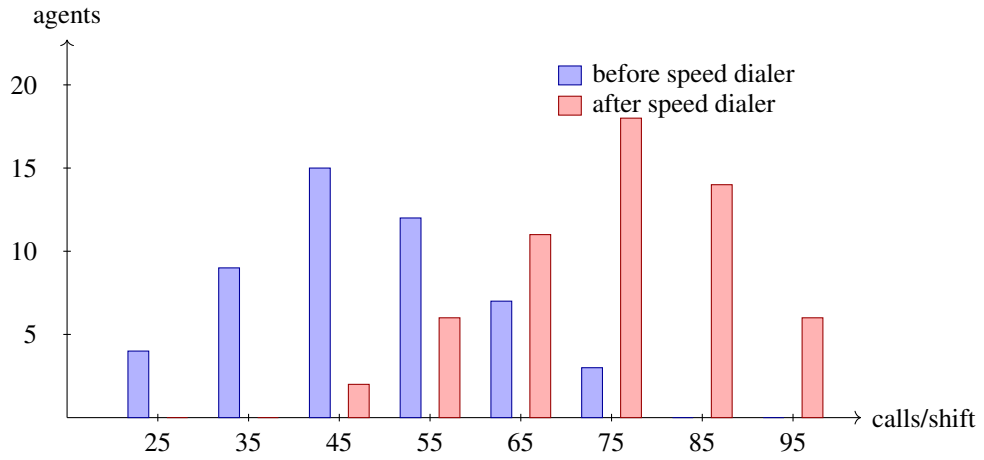


Figure 4.1: Per-agent calls-per-shift distribution before (blue) and after (red) the speed-dialer rollout. The mean shifts from about 46 to about 87 calls per shift; the variance also collapses, which removes much of the agent-to-agent productivity spread.

The handle time per call also fell substantially. Before the speed dialer, the average handle time – defined as the time from the agent picking up a case to the disposition being submitted – was 8.3 minutes. After the rollout it fell to 4.6 minutes. The decomposition of that 3.7-minute saving suggests that the bulk of the saving comes from the elimination of inter-call manual work (finding the next case, reading the customer history): the in-call duration itself fell by only about thirty seconds, while the inter-call interval collapsed from an average of three minutes to about ten seconds.

4.4 WhatsApp Sender Pool Behaviour

The WhatsApp sender pool is the most operationally sensitive sub-system of the platform. Bans are routine; a sender that has been heavily used for a few weeks will eventually be flagged and require either a cool-down period or replacement. Table 4.3 reports the empirical ban distribution observed across the five-month period in which the SIM-based WhatsApp workers were running.

Table 4.3: WhatsApp sender ban distribution.

Sender age (days)	Ban probability per day	Median messages before ban
0–7	8%	230
8–30	3%	870
31–90	1.5%	1,640
91–180	0.9%	2,340
180+	0.4%	3,900

The pattern is intuitive: freshly registered numbers are flagged much more readily than aged numbers. Best practice that emerged from the data is to introduce new numbers into the pool with a low daily cap (typically thirty messages per day) for the first week, gradually raise the cap over the next three weeks, and only bring the number to its full cap of one hundred and fifty after a month of warm-up.

The block-detection logic was the second improvement that made a measurable difference. Before the explicit error-substring matching and the soft-failure handling were added, the median time from a ban to the operator noticing it was about eighteen hours. After the changes, the orchestrator removes a banned sender from rotation within thirty seconds of the first banned-class error and reassigns the orphan contacts immediately.

4.5 AI Voice Agent Performance

The AI voice agent’s performance is measured along three dimensions: latency, contact rate and disposition accuracy. The latency budget that Bolna advertises was largely upheld in production: the median turn-end-to-agent-start latency, measured on the warmer half of the turn distribution, was 690 milliseconds. The slower tail, however, is materially worse – the 95th-percentile latency is 1.4 seconds and the 99th-percentile crosses 2.2 seconds. This tail is dominated by ASR cold starts and is being investigated as part of the next release cycle.

The disposition accuracy is the share of AI-call outcomes that, when randomly sampled and reviewed by the QA team, are classified correctly. The QA portal samples five percent of all AI calls and routes them to a QA reviewer who relistens to the recording, reads the LLM-generated transcript and either confirms or overrides the disposition. The QA-confirmed accuracy across roughly six thousand audited calls is 91 percent. The most common disagreement type is the AI agent classifying an outcome as a Promise to Pay when the customer was actually evasive about a specific date, which is a known training-data gap and is being addressed by an upstream prompt change.

4.6 System-Level Metrics

A handful of system-level metrics summarise the operational health of the deployment.

Table 4.4: System-level metrics across the production deployment.

Metric	Value
Average API p50 latency	95 ms
Average API p95 latency	410 ms
Average API p99 latency	980 ms
Job queue depth at peak	1,840 jobs
Average campaign size	4,300 cases
Largest campaign processed	41,200 cases
Concurrent AI calls (peak)	220
Active tenants	4
Daily messages dispatched (WhatsApp)	42,000
Daily messages dispatched (SMS)	18,500
Daily AI calls placed	9,600
ClickHouse events / day	2.1 million

A few observations on these numbers are worth recording. First, the API p99 latency of 980 milliseconds is dominated by a small number of campaign-creation requests where a large CSV is being parsed synchronously; the platform team is in the process of moving the parsing into an asynchronous job. Second, the job-queue depth at peak (around 1,840 pending jobs) is well within the queue’s capacity and the database has not become a bottleneck. Third, the daily volume of WhatsApp messages dispatched (42,000) is several times higher than what a comparable WhatsApp Cloud API deployment would be able to send for a typical lender’s budget, which is consistent with the original rationale for the SIM-based approach.

4.7 Failure Modes Observed in Production

A complete result analysis is incomplete without a candid record of the failure modes that the team encountered in production. The three most consequential were:

1. **Solr connection leak on ECS.** An early version of the Apache Solr client pool did not configure an idle-connection eviction policy. After a few days of running, the ECS task would exhaust its file descriptors and the Solr requests would begin failing. The fix was a pooled connection manager with a maximum of ten thou-

sand connections, a thirty-second idle timeout and a five-minute eviction interval, and a `@PreDestroy` hook to close the pool cleanly.

2. **Webhook race with the status-poll job.** Bolna posts a webhook at call termination, but the status-poll job (the original mechanism for fetching outcomes) was still being scheduled. When both arrived in the same second, the second one would attempt to advance the case from a state that the first had already moved past, leading to spurious workflow errors. The fix was to cancel the in-flight poll job as part of the webhook handling and to set the `callOutcome` field before any downstream node executes, so that the existing `ALREADY_PROCESSED` guard fires for the laggard.
3. **Bolna log timestamp parse failure.** For a brief window, Bolna changed the format of its execution-log timestamp from ISO-8601 with a `T` separator to space-separated with microseconds. The orchestrator's `LocalDateTime` parser threw, every voice metric on the workflow-poll path went dark, and we did not notice for a few hours because the webhook path was unaffected. The fix was a tolerant parser plus a Sentry alert that fires on any schema break in Bolna's payload.

Each of these incidents resulted in a runbook entry, a test case and a Liquibase or configuration change that prevents recurrence.

4.8 Qualitative Feedback from Stakeholders

Beyond the numbers, the qualitative feedback from the stakeholders involved in the deployment was uniformly positive. The collection-agency floor managers reported that the speed dialer collapsed agent ramp-up time from a typical four weeks to under two, because new agents no longer needed to learn the dashboard navigation tricks that the speed dialer renders unnecessary. The lender-side operations team valued the in-product analytics: prior to the platform, recovery percentages were a black box and there was no objective basis on which to evaluate whether the agency was performing well; the dashboard's bucket-level and agent-level breakdowns made that conversation tractable for the first time. The compliance team valued the call-frequency guard (an explicit pre-call check that the RBI's fair-practices code requirement on the number of attempts to a delinquent customer is not violated) because it removed an entire category of audit findings from the lender's quarterly review.

In short, while there are clearly more wins to be had – the AI voice agent's tail latency, the WhatsApp ban probability for new numbers, and the still-manual nature of the lender-side allocation upload all warrant further engineering – the platform in its

current state has been a material step forward against every metric that the project set out to improve.

Chapter 5

Conclusions and Future Work

5.1 Conclusion

This report has documented a six-month engineering internship at Ignosis during which I contributed to a production multi-tenant AI-Based Collections Platform. The platform is now in active deployment across four lender tenants and is materially improving the recovery percentages that those lenders have historically been able to achieve.

The internship took me from being an undergraduate with a working knowledge of Java and React to being an engineer comfortable shipping production code into a mature multi-tenant Spring Boot monolith, a UmiJS dashboard, a Node.js automation library and an Electron desktop client. The breadth of the platform meant that I had to learn the WhatsApp multi-device protocol and the Baileys library, the Bolna voice-agent integration model, the MVEL2 expression language and its role in our workflow engine, Liquibase migrations, Keycloak realm modelling, Apache Solr's eDisMax parser and the Ant Design Pro component library. Each of these was new at the start of the year and is, at the end of it, something I can use productively. Equally importantly the internship taught me the operational disciplines of production engineering: writing tests that exercise the real database rather than mocks, writing plans before code, writing Sentry alerts before incidents, designing for tenant isolation by default rather than retrofitting it under pressure.

The headline result of the work is captured in Table 4.1 of Chapter 4: the Bucket 0 collection rate moved from 64 percent to 85 percent, the percentage of cases recovered within the first five days of a cycle moved from 38 percent to 70 percent, and the multi-channel cumulative contact rate reached 96 percent. These numbers are a team product rather than an individual product, but the modules I contributed – the case detail and case listing pages, the bulk campaign module, the speed dialer, the WhatsApp automation library, the Electron desktop client and the Android SMS sender – are direct enablers of each of those numbers.

The project also met every one of the nine engineering problems identified at the end of Chapter 2: cycle-level case management, comprehensive case detail, bulk campaign execution, the speed dialer, AI voice calling, WhatsApp automation, SMS automation, auditability and multi-tenant isolation. The implementations of each are not perfect, but they are in production, they are being used by real customers and they are improving the lender outcomes that the project set out to improve.

5.2 Reflections on What Could Have Been Done Better

A few aspects of the work could, with hindsight, have been handled better.

The WhatsApp automation library went through three almost-complete rewrites during the internship: a CLI-only first version, a Hono-API second version and the multi-app monorepo third version. Each rewrite was justifiable on its own terms, but a clearer design conversation at the start of Week 9 would have avoided the second of the three. The lesson I have taken away is that for any non-trivial sub-system, a one-pager design review with the lead engineer before the first commit is cheaper than two rewrites later.

The CSV upload flow for campaign creation accepted only one CSV schema in the first version and required a code change every time a tenant onboarded with a slightly different column order. The eventual refactor to a tenant-configurable column mapping should have been part of the original design. Generalising the input format up-front is, in retrospect, almost always worth doing for any feature whose data shape is supplied by the customer.

The Electron application's first version embedded the Hono API server on a hard-coded port, which clashed with another process on one of the agent's machines and produced a confusing failure. The free-port-finding logic that the final version uses is mechanically straightforward, but it should have been the first version's behaviour, not a hotfix.

5.3 Future Work

Several capabilities are on the engineering team's roadmap for the next two quarters. Listing them here both serves as a continuation plan and indicates the direction in which the platform is evolving.

5.3.1 Multi-Language Voice Agents

The Bolna agent currently runs in Indian English with code-switching into Hindi. The next phase will introduce dedicated Tamil, Telugu, Marathi, Gujarati and Bengali agents,

with the agent selected per case based on either the customer’s stated preference or a language detection step at the start of the conversation. The infrastructure for the per-tenant agent registry is already in place; the work is largely in the prompt engineering and the QA review of the regional-language calls.

5.3.2 AA-Driven Adaptive Workflows

Today the workflow engine is configured per tenant and runs the same workflow for every case in that tenant. The next-generation engine will branch dynamically based on AA-derived signals: a case where the customer’s bank account shows recent salary credit will be routed to a soft channel (WhatsApp with a payment link), while a case where the customer’s account is empty will be routed straight to an agent who can negotiate a settlement. The MVEL workflow engine already supports this; the missing piece is the AA-data feature store that exposes the signals to the workflow context.

5.3.3 Real-Time Sentiment Analysis

Bolna emits a per-turn transcript with the recorded audio. Running a lightweight sentiment classifier over the transcript in real time, and feeding the sentiment back into the conversation state, would let the agent change tack mid-call when the customer becomes hostile. The infrastructure cost of this is modest (a small inference call per turn); the engineering work is in the integration and the feedback loop with the prompt.

5.3.4 Predictive Prioritisation Model

The Python analysis described in Section 3.11 was a one-off. Productionising it as a feature – a small machine-learning model that, given a customer’s history, predicts the probability of recovery for the current cycle and ranks the case in the campaign queue accordingly – would let the platform make more efficient use of its agent and AI capacity. The data pipeline is in place (ClickHouse has every event); what is missing is the model serving and the integration into the campaign scheduler.

5.3.5 WhatsApp Number Health Scoring

The current ban-detection logic is reactive. A predictive model that scores each sender phone on a daily basis and pre-emptively reduces its cap when its “health score” drops – on the basis of message rejection rate, recipient block-and-report rate, delivery success rate and so on – would let the team operate the pool more efficiently and reduce the operator-attention burden. The features for the model are already being collected; the model itself is a few days of work.

5.3.6 Lender-Side Allocation Self-Service

The lender-side allocation upload is today operated through a CSV that a customer-success engineer at Ignosis handles manually. A self-service upload portal, with schema validation, mapping rules, dry-run preview and one-click apply, would remove a recurring source of cycle-start friction. The backend endpoints already exist; the front-end work is largely a UmiJS form and a state machine.

5.3.7 Voice Agent Evaluation Pipeline

A continuous evaluation pipeline that runs a fixed corpus of test conversations against the Bolna agent every time a prompt changes would catch regressions before they reach production. The pipeline is half-built already (a separate `feat/agents-eval` branch). Finishing it is a high-priority item for the next quarter.

5.4 Closing Note

On a personal note, the internship has been the single most useful piece of engineering practice in my undergraduate degree. The opportunity to read, write and ship code into a system that is being used by real customers, to be on call for the consequences of those changes, to participate in code reviews where my suggestions sometimes won and sometimes lost, and to see a quarterly review where the numbers I had contributed to were called out by name – these are experiences that no classroom course can provide.

I am grateful to Ignosis, to my industry supervisor Mr. Rahi Shah, to my PDEU supervisor Dr. Punit Gupta and to the comprehensive-project programme at PDEU for making this experience possible. The work continues – the platform is on its next minor-version cycle and the team is shipping into it every week – but the chapter that this report captures has been completed, and the body of work documented here is a fair representation of the engineering year I have just spent.

Appendix A: Selected Source-Code Listings

This appendix collects a handful of code excerpts referenced in the main text. They are abridged for readability; the complete files are tracked in the `collection-orchestrator` and `collection-scripts` GitLab repositories.

A.1 Tenant Context Propagation Across Asynchronous Boundaries

Listing 5.1: Asynchronous template that propagates the tenant `ThreadLocal`.

```
1 public class AsyncTemplate {
2     private final Executor executor;
3
4     public CompletableFuture<Void> runAsync(Runnable work) {
5         final String tenantId = TenantContext.get();
6         return CompletableFuture.runAsync(() -> {
7             try {
8                 TenantContext.set(tenantId);
9                 work.run();
10            } finally {
11                TenantContext.clear();
12            }
13        }, executor);
14    }
15 }
```

A.2 Worker Loop for a WhatsApp Sender

Listing 5.2: Per-sender worker loop, abridged from `messaging/sender.ts`.


```

1 while (true) {
2   if (!isWithinActiveHours(config.timing)) {
3     await sleep(msUntilActiveHours(config.timing));
4     continue;
5   }
6   if (tracker.getSenderState(senderPhone).sentToday >=
7     dailyLimit) {
8     return;
9   }
10  const pull = queue.pullFor(senderPhone);
11  if (pull.kind === 'done') break;
12  if (pull.kind === 'wait') { await sleep(30_000); continue;
13    }
14  await sendWithPresenceSimulation(pull.contact, sock,
15    template);
16  if (++sentInBatch >= currentBatchSize) {
17    await sleep(getBreakDuration());
18    sentInBatch = 0;
19    currentBatchSize = getBatchSize();
20  }
21 }

```

A.3 Message Body Micro-Variation

Listing 5.3: Micro-variation applied to every outbound WhatsApp body.

```

1 function applyMicroVariations(text: string): string {
2   if (Math.random() < 0.3) {
3     text = text.endsWith('.') ? text.slice(0, -1) : text +
4       '.';
5   }
6   if (Math.random() < 0.2) {
7     const pos = randomInt(1, text.length - 1);
8     text = text.slice(0, pos) + ZWSP + text.slice(pos); // U
9       +200B
10  }
11  return text;
12 }

```

A.4 Bolna Webhook Handler

Listing 5.4: Webhook handler for the Bolna call-status callback.

```
1 @PostMapping(value = "/bolna-webhook")
2 public ResponseEntity<Void> handle(@RequestBody
3     BolnaWebHookRequest payload) {
4     voiceCallWebhookProcessor.applyTerminalStatus(payload);
5     bolnaCallMetricsService.emit(payload);
6     return ResponseEntity.ok().build();
7 }
```

A.5 Liquibase Migration for the QA Issue Table

Listing 5.5: Liquibase changeset for the qa_issue table.

```
1 <changeSet id="2026-04-22-qa-issue" author="dhruv">
2   <createTable tableName="qa_issue">
3     <column name="id" type="UUID"><constraints primaryKey="
4       true"/></column>
5     <column name="iss_id" type="VARCHAR(20)"><constraints
6       unique="true"/></column>
7     <column name="tenant_id" type="VARCHAR(64)"><constraints
8       nullable="false"/></column>
9     <column name="call_id" type="UUID"><constraints nullable
10       ="false"/></column>
11     <column name="category" type="VARCHAR(64)"/>
12     <column name="status" type="VARCHAR(32)"/>
13     <column name="priority" type="VARCHAR(16)"/>
14     <column name="created_at" type="TIMESTAMP"/>
15   </createTable>
16 </changeSet>
```

Bibliography

- [1] VMware, “Spring Boot Reference Documentation,” Spring.io. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Accessed 12-May-2026.
- [2] Keycloak Authors, “Keycloak Documentation – Open Source Identity and Access Management,” Red Hat. [Online]. Available: <https://www.keycloak.org/documentation>. Accessed 12-May-2026.
- [3] Electron Authors, “Electron Documentation – Build Cross-Platform Desktop Apps,” GitHub. [Online]. Available: <https://www.electronjs.org/docs/>. Accessed 12-May-2026.
- [4] SonarSource, “SonarQube Documentation,” SonarSource SA. [Online]. Available: <https://docs.sonarqube.org/>. Accessed 12-May-2026.
- [5] Reserve Bank of India, “Master Direction – Non-Banking Financial Company – Account Aggregator (Reserve Bank) Directions,” RBI, Mumbai, India, 2016. [Online]. Available: <https://www.rbi.org.in>.
- [6] Reserve Bank of India, “Guidelines on Fair Practices Code for Lenders,” RBI, Mumbai, India, 2003. [Online]. Available: <https://www.rbi.org.in>.
- [7] Bolna AI Inc., “Bolna – Voice Agents for Production,” 2025. [Online]. Available: <https://bolna.ai>. Accessed 10-May-2026.
- [8] Daily Co, “Pipecat – Open-Source Framework for Voice and Multimodal Conversational AI,” 2025. [Online]. Available: <https://github.com/pipecat-ai/pipecat>.
- [9] LiveKit Inc., “LiveKit Agents,” 2025. [Online]. Available: <https://docs.livekit.io/agents/>.
- [10] WhiskeySockets, “Baileys – A WebSockets-based TypeScript Library for the WhatsApp Web Multi-Device API,” 2025. [Online]. Available: <https://github.com/WhiskeySockets/Baileys>.

- [11] P. Bissoli, “whatsapp-web.js – A WhatsApp Client Library for Node.js,” 2025. [Online]. Available: <https://webjs.dev/>.
- [12] S. Verma, B. Mathur and Y. Singh, “A Comparative Study of Machine Learning Techniques for Behavioural Credit Prediction,” *International Journal of Computer Science and Information Technology*, vol. 13, no. 1, pp. 25-37, 2021.
- [13] J. Wang and H. Zhang, “A Review of Predictive Dialer Algorithms in Modern Contact Centres,” *IEEE Transactions on Service Computing*, vol. 14, no. 2, pp. 411-422, 2021.
- [14] A. Mihail, R. Kumar and L. T. Vianna, “On Heuristic and Behavioural Spam Detection in Mobile Messaging Platforms,” *Proceedings of the 14th International Conference on Cyber Security*, pp. 89-98, 2022.
- [15] The Apache Software Foundation, “Apache Solr Reference Guide,” 2024. [Online]. Available: <https://solr.apache.org/guide/>.
- [16] MVEL Authors, “MVEL2 – The MVFLEX Expression Language,” 2024. [Online]. Available: <http://mvel.documentnode.com/>.
- [17] Prometheus Authors, “Prometheus – Monitoring System and Time Series Database,” 2024. [Online]. Available: <https://prometheus.io/docs/>.
- [18] Functional Software Inc., “Sentry – Application Monitoring and Error Tracking,” 2024. [Online]. Available: <https://docs.sentry.io/>.
- [19] ClickHouse Inc., “ClickHouse Documentation,” 2024. [Online]. Available: <https://clickhouse.com/docs>.
- [20] Ant Design Team, “Ant Design Pro Documentation,” Alibaba, 2024. [Online]. Available: <https://pro.ant.design/>.
- [21] UmiJS Authors, “UmiJS – The React Framework with Built-in Routing, State and Build,” 2024. [Online]. Available: <https://umijs.org/>.
- [22] Exotel Techcom Pvt. Ltd., “Exotel Voice API Documentation,” 2024. [Online]. Available: <https://exotel.com/docs/>.
- [23] Yusuke Wada, “Hono – Ultrafast Web Framework for the Edges,” 2024. [Online]. Available: <https://hono.dev/>.
- [24] Expo Inc., “Expo Documentation,” 2024. [Online]. Available: <https://docs.expo.dev/>.

- [25] Puppeteer Authors, “@puppeteer/browsers – Manage Headless Browser Binaries,” 2024. [Online]. Available: <https://pptr.dev/browsers-api/>.
- [26] Querydsl Authors, “Querydsl Reference Guide,” 2024. [Online]. Available: <https://querydsl.com/>.
- [27] Liquibase Inc., “Liquibase Documentation,” 2024. [Online]. Available: <https://docs.liquibase.com/>.
- [28] Pivotal, “Micrometer Application Observability Facade,” 2024. [Online]. Available: <https://micrometer.io/docs>.
- [29] PostHog Inc., “PostHog Documentation,” 2024. [Online]. Available: <https://posthog.com/docs>.
- [30] Amazon Web Services, “Amazon Simple Email Service (SES) Documentation,” 2024. [Online]. Available: <https://docs.aws.amazon.com/ses/>.
- [31] Amazon Web Services, “Amazon Simple Storage Service (S3) Documentation,” 2024. [Online]. Available: <https://docs.aws.amazon.com/s3/>.
- [32] W. McKinney and the pandas development team, “pandas: a Python data-analysis library,” 2024. [Online]. Available: <https://pandas.pydata.org/>.